CODEBASE AUDIT & PRODUCTION PLAN · 04 SEP 2026

VaadVivaad: from a working demo to a system that can be trusted

The prototype does something genuinely novel — it stages an adversarial legal debate over retrieved Indian criminal law. It is also, today, one blocking event loop, one shared mutable dict, and one hallucination feedback loop away from being unshippable. This is the read, the ledger, and the route.

SURFACE ~3.1k lines, app code	SERVICES 4 compose stack	GEMINI CALLS 12 per debate, serial
DEBATE DEPTH 3 turns, 1 round	TESTS 0 no CI	AUTH'D LLM ROUTES 0 / 10 open to internet

§ 00

What this system actually is

Before proposing anything, the honest description of the current machine — because most of what follows is a consequence of it.

A user types an incident description and a free-text evidence list. The backend hands that to Gemini to be classified as legal / non-legal and rewritten into formal language, extracts a **single** IPC section, looks for one precedent in Qdrant, pulls a section description and a typical-evidence list, then runs two personas — plaintiff counsel and defence counsel — for exactly one round. Three arguments total, each a single JSON object with `point`, `evidence`, `demand`. The transcript streams to the browser over Socket.IO and can be saved to MongoDB.

01 **Refine & gate** 1 LLM call	02 **Precedent search** 1 embed call	03 **IPC section** 1 LLM call	04 **Similar case** 1 LLM + 1 embed	05 **Section detail** 1 LLM + 1 embed
06 **Evidence types** 1 LLM + 1 embed	07 **Opening (plaintiff)** 1 LLM call	08 **Rebuttal (defence)** 1 LLM + 2s sleep	09 **Reply (plaintiff)** 1 LLM + 2s sleep	

STEPS 04–06 ARE MUTUALLY INDEPENDENT AND RUN SEQUENTIALLY ANYWAY. STEPS 03–06 ONLY FIRE WHEN QDRANT RETURNS NOTHING — WHICH IS ALWAYS, ON A FRESH INSTALL.

THE READ

The *idea* is the asset: adversarial argument is a genuinely better interface for legal reasoning than a chatbot, because it surfaces the counter-case a single-voice assistant hides. Nothing in this plan touches that.

The *implementation* has three structural problems that no amount of feature work fixes around. It blocks the event loop on every LLM call, so the server serves one debate at a time. It keeps session state in a module-level dict, so it cannot run two replicas. And when vector search comes back empty, it asks Gemini to invent “real, verifiable” case law and **writes that invention back into the shared vector store**, where the next user retrieves it as ground truth. Fix those three and everything else is additive.

§ 01

Findings, ranked by what they cost you

All verified against the source. File references are where the behaviour lives, not where it is described.

F-01

Hallucinated case law is persisted into the shared vector store CRITICAL

When `find_similar_cases()` returns empty, the flow asks Gemini for three “real and verifiable” Indian judgments, then `saveSimilarCases()` writes the first one into the `case_laws` collection. Same pattern for `saveIPCSection()` and `saveIPCEvidence()`. Collections ship empty, so this path is the *only* path in practice.

The consequence compounds: fabricated precedent becomes retrieved precedent, which the next debate cites with a similarity score attached, which reads to a user as corroboration. A legal product that manufactures its own citations is not shippable at any scale.

`SIMILAR_CASES_MAIN.py:11 · saveSimilarCases.py · user.py:126`

F-02

One user's case text leaks into every other user's retrieval CRITICAL

The generated “similar case” is derived from the user's own incident description and lands in a global, un-namespaced Qdrant collection queried by all users. Real filings contain names, locations, and allegations. There is no tenant key, no consent step, and no deletion path.

`saveSimilarCases.py · find_similar_case.py:COLLECTION_NAME`

F-03

Every LLM call blocks the shared event loop CRITICAL

`run_debate_flow` is a coroutine, but every call inside it — `client.models.generate_content()`, the Qdrant search, and `time.sleep(RETRY_DELAY)` in the retry loop — is synchronous. For the 30–50 seconds a debate takes, the process serves nothing else: not health checks, not logins, not other debates.

This is why the app feels single-user. It *is* single-user.

`model_config.py:24,34 · user.py:88`

F-04

Debate sessions are unauthenticated and joinable by guess CRITICAL

`POST /user/start-case` has no auth dependency. The session id is a client-generated 10-digit number. The Socket.IO `connect` handler authenticates nothing, and `join_room` accepts any session id and subscribes the caller to that room's stream. Anyone can enumerate rooms and read other people's case debates in real time.

`user.py:47,61,283 · Case.jsx:29`

F-05

Ten unauthenticated endpoints that each spend money CRITICAL

The entire `/gemini/*` router — argument generation, section lookup, summary, evidence — is public, unmetered, and unrate-limited, as is `/vectordb/*`. A single script drains the API quota, or the bill, in minutes. There is no rate limiter anywhere in the codebase.

`gemini.py (all routes) · vectordb.py (all routes)`

F-06

The evidence the user types is never used HIGH

`/start-case` stores `evidence` into `case_data_store` and `run_debate_flow` reads only `incident_description`. The form marks the field `required`. Users are being asked to write the single most case-specific input in the product, and it is dropped on the floor — the debate instead argues over a generic “typical evidence for this section” list.

`user.py:288 (write) · user.py:82 (read, omits it)`

F-07

The case-analysis panel never renders HIGH

The frontend gates its round-0 “Case analysis / IPC section / precedent” card on a `case_details` socket event. The backend never emits it — the string does not exist server-side. Every debate therefore falls through to the plain argument card, and the retrieval work the system just paid for is invisible to the user.

`Case.jsx:151,158 · no emitter in app/`

F-08

The whole system is written against a repealed statute HIGH

Every prompt, collection, schema and field name is IPC. The Bharatiya Nyaya Sanhita replaced the IPC for offences committed from 1 July 2024, with BNSS and BSA replacing the CrPC and the Evidence Act. For any recent incident the product returns a section number that no longer exists. This is a correctness defect in the core domain, not a roadmap nice-to-have.

`domain-wide: prompts/, schemas/, collection names`

F-09

Retrieval is a single word against `k=1` HIGH

Precedent search embeds `processed_prompt["crime_type"]` — one word, e.g. “murder” — takes the top 1 hit, hard-filters `metadata.court` against a fixed four-value list, and drops anything under 0.70 cosine. A one-word query cannot discriminate between two murder cases with opposite facts. Separately, `search_by_ipc_section` pays for an embedding to do what is a primary-key lookup.

`user.py:112 · find_similar_case.py:28 · search_by_ipc_section.py:38`

F-10

The debate has no memory of itself HIGH

`debate_history` is accumulated for the transcript but never fed back into a prompt. Each turn sees only the single immediately preceding argument, and the prompts cap output at exactly one point. With one round, the “debate” is three turns that cannot reference anything earlier, cannot concede, and cannot build. There is also no judge: the UI titles the final card “Final verdict” and renders a history dump.

`user.py:184,213 · argument_prompt.py · MessageCard.jsx:33`

F-11

Session state is a module-level dict HIGH

`case_data_store = {}` lives in the process. Two replicas cannot share it, a restart drops every in-flight debate, and a client that connects to a different worker than the one that took its POST gets “Session data not found.” This single line is what pins the deployment to one instance.

`user.py:38`

F-12

User text is interpolated raw into every prompt HIGH

Every prompt builder is an f-string with the user's words dropped inside. The only guard is a single LLM classifier asked to return `{"status":"non-legal"}` — trivially steered by instructions inside the same input it is judging. There is no delimiter discipline, no instruction-hierarchy separation, no output validation against injected content.

`user_input_prompt.py:5 · all of prompts/`

F-13

Auth returns 200 for failure, and enumerates users MEDIUM

Login with bad credentials returns HTTP 200 with `{"status":"fail"}`; signup with an existing email returns a distinguishable 200 that confirms the address is registered. There is no unique index on `users.email`, so concurrent signups can both pass the existence check. Tokens last 30 minutes with no refresh, and the frontend keeps `isLoggedIn` in localStorage without ever revalidating — so the UI stays “logged in” while every API call 401s.

`auth.py:21,35 · authStore.js:29 · ProtectedRoute.jsx`

F-14

JSON is coaxed by prompt, then salvaged by string surgery MEDIUM

`GENERATE_RESPONSE_FROM_GEMINI` strips markdown fences by hand and `json.loads`es the remainder; any failure returns `None`, which propagates as a `TypeError` several frames away. The Gemini SDK supports `response_schema` with a Pydantic model and `response_mime_type="application/json"` — structured output makes the entire cleaning block and most of the failure surface disappear.

`model_config.py:42–64`

F-15

No observability, no tests, no indexes MEDIUM

`print()` is the logging strategy — no structure, no request id, no way to answer “why was this debate slow.” No test files exist and no CI runs. MongoDB has no index on `debates.user_id` (collection scan per dashboard load) or `users.email`. The frontend ships MUI, Radix, FontAwesome, react-icons and Lucide simultaneously, plus `jspdf`, `html2canvas` and `react-to-print` that nothing imports.

repo-wide · package.json

§ 02

Keep, modify, replace, remove

The explicit disposition of every meaningful piece, so nobody rewrites something that is already right.

COMPONENT	CALL	REASONING
Adversarial debate concept	KEEP	The differentiator. Deepen it — don't dilute it into a chatbot.
Design system `tailwind.config, index.css`	KEEP	Fraunces / IBM Plex, brass-on-parchment, docket labels. Genuinely good and on-subject. Extend it, don't restyle.
Qdrant + Gemini embeddings	KEEP	Right tool, correct asymmetric task types (`RETRIEVAL_DOCUMENT` / `RETRIEVAL_QUERY`). The problem is what gets written into it, not the store.
Compose stack & container hardening	KEEP	Non-root users, healthchecks, no-default secrets, resource limits. Above the bar for this stage.
Socket.IO transport	KEEP	Correct for a multi-actor, server-driven stream. Add the Redis manager and auth, keep the protocol.
Debate orchestration `user.py:75–268`	MODIFY	Extract from the route layer into a state-machine service. Async, parallel fan-out for steps 04–06, resumable, persisted per turn.
Prompt layer `controller/prompts/`	MODIFY	Keep the personas; move to versioned templates with delimiter-fenced user input and `response_schema` instead of hand-written JSON blocks.
Retrieval	MODIFY	Query with the full refined description, not a one-word crime type. `k=8` → rerank → top 3. Drop the hard court filter to a boost.
Auth	MODIFY	Correct status codes, uniform failure messages, unique index, refresh tokens, socket handshake auth, server-side session revocation.
Ingest-on-generate `save*.py called from controllers`	REMOVE	The hallucination loop (F-01/F-02). Model output must never write to the retrieval corpus. Keep the `save*` helpers, but call them only from an offline ingest job over real sources.
`case_data_store` **dict**	REPLACE	Redis with TTL, keyed by a server-issued opaque session id bound to the user.
Blocking Gemini client	REPLACE	`client.aio.models.generate_content` + `asyncio.gather` , or move generation to a Celery/ARQ worker and keep the web tier for I/O only.
Hand-rolled JSON repair	REPLACE	Structured output with Pydantic schemas. Deletes ~25 lines and a whole class of runtime failure.
`print()` **logging**	REPLACE	`structlog` JSON to stdout + OpenTelemetry spans per debate step.
Public `/gemini/*` **router**	REMOVE	Ten public money-spending endpoints that exist only as manual test harnesses. Delete, or move behind auth + an admin scope.

COMPONENT	CALL	REASONING
`ArgumentCard.jsx`, **`CaseSubmissionForm.jsx`**	REMOVE	Dead components; `ArgumentCard` reads `argument.elements`, a shape nothing produces.
MUI + Radix + FontAwesome + react-icons	REMOVE	Four UI/icon systems for a Tailwind app that renders Lucide. Also drop unused `jspdf` / `html2canvas` / `react-to-print` until export ships (§03 F-08).
Nested git repos	REPLACE	Two independent `.git` dirs inside an outer repo. Make it one monorepo (or real submodules) before CI, which cannot reason about the current shape.

§ 03

Ten features, built on the debate

Each one extends the adversarial core rather than bolting a second product beside it. Impact is user-visible value; complexity is engineering cost against the current codebase.

01 The Bench — a presiding judge that actually rules

Fixes F-10 · closes the loop the UI already promises

After the rounds close, a third persona with a distinct system prompt reads the full transcript and issues a structured ruling: which arguments survived, which were abandoned, the legally decisive issue, a disposition, and — critically — *a confidence band and the specific facts that would flip it*. Rendered as a sealed order the user opens.

This is the highest impact-to-effort item in the plan. One prompt, one persona, one schema; it converts a transcript into an answer, and it makes every earlier turn feel consequential. The UI already has a “Final verdict” card waiting for it.

IMPACT ■■■■■ COMPLEXITY ■□□□□

02 Verified citations, or none at all

Fixes F-01 · the credibility precondition for everything else

Invert the trust direction. Ingest a real corpus offline — the Supreme Court's judgment portal, eCourts, India Code — into `case_laws` with a canonical id per judgment. All free, official publishers; no commercial legal database is needed. At generation time the model may only cite from a shortlist of retrieved documents passed in context, and every citation in the output is checked against that shortlist before the turn is emitted. Unmatched citations get stripped and the turn is marked `UNSUPPORTED` in the UI rather than silently shown.

Every cited case in the transcript becomes a link to the actual judgment with the matched passage highlighted. That single interaction is what separates a demo from a tool a law student will use twice.

IMPACT ■■■■■ COMPLEXITY ■■■■□

03 Real proceedings: multi-round, streamed, interruptible

`Fixes F-03, F-10 · turns 3 static turns into a session`

Structured phases modelled on an actual hearing — opening statements, examination of the evidence, rebuttal, closing — with the number of rounds set by case complexity rather than a hard-coded `range(1, 2)`. Each turn streams token-by-token over the existing socket, so the artificial `sleep(2)` can go: the typing indicator becomes real text arriving.

Add a **Pause** and an **Object** control. Objecting lets the user interject a fact or challenge mid-round, and the affected counsel must respond to it in the next turn. That is the moment the product stops being a video and becomes an instrument.

IMPACT ■■■■■ COMPLEXITY ■■□□□

04 Case file intake — FIR, chargesheet, notice

`Fixes F-06 · replaces two textareas with the documents people actually hold`

Upload a PDF or a phone photo of an FIR, chargesheet, legal notice or bail order. Parse it (Gemini reads PDFs and images natively; Docling runs locally for hard scans), extract a structured case object — parties, dates, sections charged, seizure list, witness names — present it for correction, then run the debate over that instead of free text.

This also finally makes the user's evidence load-bearing: each argument cites specific exhibits from the file, and the defence gets to attack the actual chain of custody rather than a generic “typical evidence” list.

IMPACT ■■■■■ COMPLEXITY ■■■□□

05 Dual-code engine: IPC ↔ BNS / BNSS / BSA

`Fixes F-08 · the domain-correctness fix`

Offences from 1 July 2024 fall under the Bharatiya Nyaya Sanhita; earlier ones stay on the IPC, and both will be live in the courts for years. Ask for the incident date, route to the correct code, and always show the concordance — IPC 302 → BNS 103, and so on across the full mapping into BNS's 358 sections.

The mapping must be a curated table checked against the official concordance and stored in Mongo, *never* asked of the model. This is also a real moat: a correct, maintained IPC↔BNS mapping with procedural (BNSS) and evidentiary (BSA) equivalents is exactly the boring asset competitors skip.

IMPACT ■■■■□ COMPLEXITY ■■■□□

06 Evidence gap analyser

Fixes F-06 · the most actionable output in the product

Diff what the user has against what the section's elements of offence require, and return a ranked checklist: *this element is unproven; here is the evidence class that usually proves it; here is who can obtain it and under which BNSS provision.* Each gap is scored by how hard the defence leaned on it during the debate — the transcript itself becomes the ranking signal.

Cheap to build (the section schema already carries `elements_of_offense` and `proof_requirement`) and it is the one screen a user would screenshot and send to their lawyer.

IMPACT ■■■■□ COMPLEXITY ■□□□□

07 Take a side — you argue, the AI opposes

The training-tool product hiding inside the demo

Flip the user from spectator to counsel. They pick a side and write the argument; the opposing AI rebuts in character and the Bench scores each exchange against named criteria — legal accuracy, use of precedent, responsiveness to the point actually made, procedural form. Ratings, streaks, and a moot-court mode with saved problems.

Same engine, different seat. It is the version a law college pays for, and it turns a one-shot tool into something with a reason to return weekly.

IMPACT ■■■■□ COMPLEXITY ■■■□□

08 Export the record

The retention hook · dependencies are already installed

One button turns a concluded debate into a formatted case brief: facts, sections invoked, arguments for and against, precedent cited with links, evidence gaps, the Bench's ruling. PDF and DOCX, on a proper Indian court-document template, with a standing not-legal-advice notice and a generation timestamp.

`jspdf`, `html2canvas` and `react-to-print` are already in `package.json` and unused — though server-side rendering (WeasyPrint or Typst) will produce a far better document than a canvas screenshot.

IMPACT ■■■■□ COMPLEXITY ■□□□□

09 Calibrated case-strength meter

`Judgement, expressed honestly`

A strength score per side, decomposed into what drives it — element coverage, precedent alignment, evidence admissibility under the BSA, procedural exposure — with a sensitivity view: *secure the CCTV and prosecution strength moves from 0.42 to 0.61.*

The discipline that makes this defensible rather than decorative: derive the score from the structured debate outcome, never let the model emit a bare number, show the decomposition always, and back-test against the labelled verdicts in the ingested corpus so the stated confidence is calibrated rather than asserted.

IMPACT ■■■■□ COMPLEXITY ■■■■□

10 Argue in the user's language

`The market this product is named for`

Accept the incident description in Hindi, Marathi, Bengali, Tamil or Telugu — typed, or spoken. Reason and retrieve in English against the corpus, then render the transcript and the brief in the input language, keeping section numbers, case citations and terms of art in their canonical form.

A person describing a police matter is not going to do it in a second language. This is the difference between a portfolio project and something usable in a district court corridor.

IMPACT ■■■□□ COMPLEXITY ■■□□□

§ 04

Workflow, UX, automation, intelligence

Improvements to the flow that already exists — before any new feature lands.

The intake is doing the least work in the product

Two required textareas and a submit button is the entire on-ramp, and a user who writes three words gets the same treatment as one who writes three paragraphs. Replace it with a short guided intake — what happened, when, who was involved, what you have — where each answer is optional but visibly improves a live **case readiness** indicator. Show the extracted structure back before spending a single token: sections identified, parties, dates, evidence classified. Let the user correct it. The model is far more accurate on a confirmed structure than on raw prose, and the correction step is also the consent moment.

Make the retrieval visible

The system currently does real work — classification, section extraction, precedent search, evidence lookup — and shows the user a spinner and three cards, with the case-analysis panel broken outright (F-07). Emit a proper `case_details` event and render the reasoning as it happens: the section identified with its elements, the precedent found with its similarity and a link, the evidence classes matched. Perceived latency drops even when wall time doesn't, and the user learns to trust the output because they watched it get built.

TODAY	TARGET
• Submit → spinner → 30–50s → three cards	• Submit → structure confirmed → streamed turns with live retrieval
• Failure disconnects the socket; only a page reload recovers	• Per-step retry; a failed turn resumes, the session survives
• Non-legal input is a dead end with no path forward	• Off-topic input gets a redirect and an example, not a disconnect
• Save is all-or-nothing at the very end	• Autosave per turn; every debate is resumable by URL
• Transcript is inert text	• Click any citation, argument or section to expand its source

Automation worth having

- **Nightly corpus ingest** — a scheduled job pulls new judgments, embeds and upserts them, and records provenance per document. The only writer to the vector store.
- **Prompt regression suite** — 40–60 fixed cases with expected sections and expected retrieval, run in CI on every prompt change. Prompts are code; they need a diff that fails.
- **Automatic case titles and tags** — the dashboard currently shows `"Unknown Case"` whenever the shape doesn't match. Derive the title from the confirmed structure at save time, not by guessing at read time.

- **Follow-up digest** — when a newly ingested judgment materially affects a saved case, notify the owner. Recurring value from work already done.

Where the intelligence should go

Not more model — better structure around it. A **complexity router** that sends trivial classification to Flash-Lite and the Bench's reasoning to a stronger tier. A **self-critique pass** where each counsel drafts, checks its own citations against the retrieved shortlist, and revises once before emitting. And **debate-aware retrieval**: when the defence raises consent, fire a targeted search for consent precedent under that section rather than reusing the opening query. Retrieval should follow the argument, not precede it once.

§ 05

Target architecture

Same four services at the edges. The change is a worker tier, a state store, and a firm rule about who may write to the corpus.

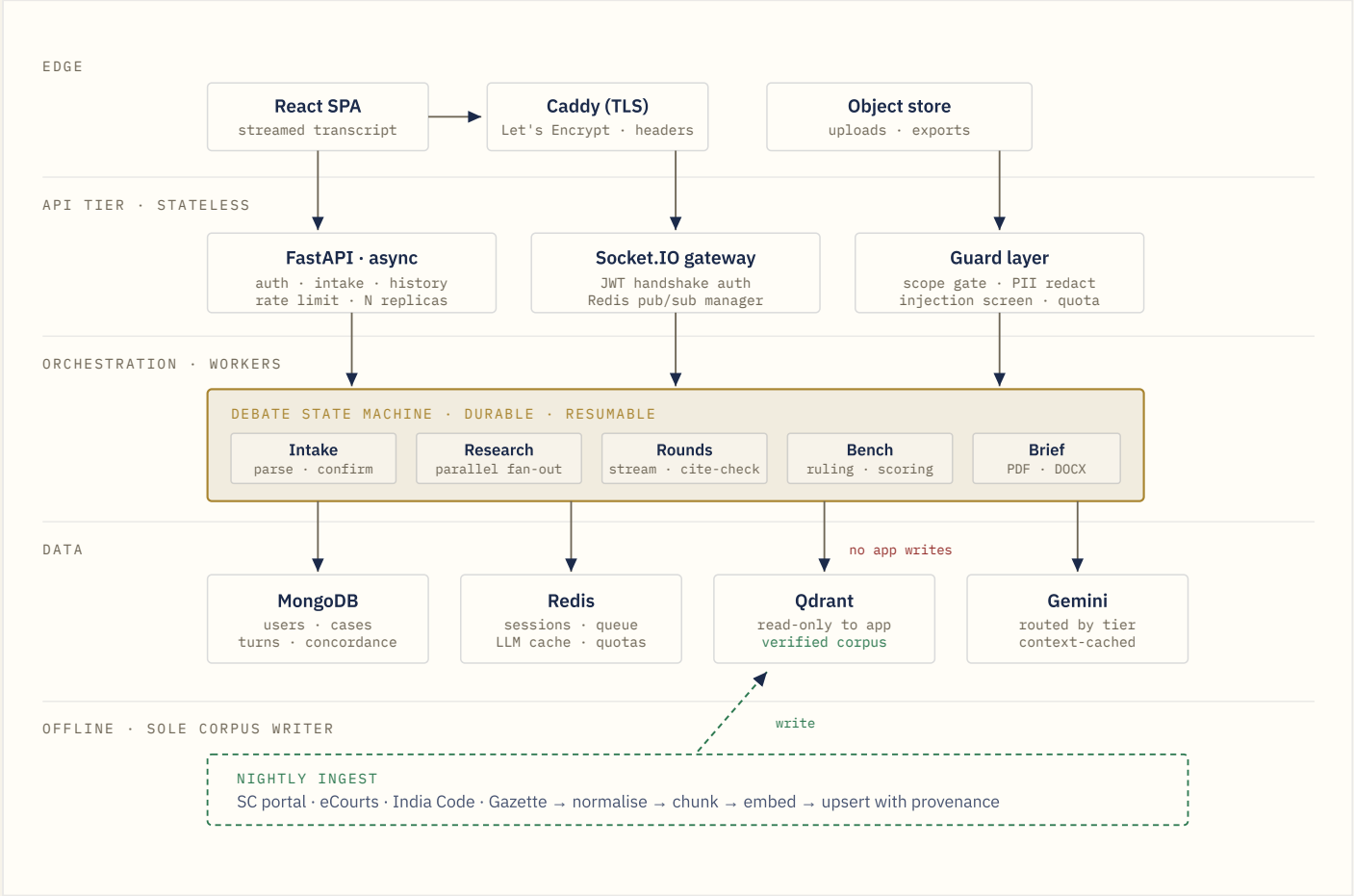

FIG. 1 — THE SINGLE MOST IMPORTANT LINE IN THIS DIAGRAM IS THE ONE THAT ISN'T THERE: NOTHING IN THE REQUEST PATH WRITES TO QDRANT.

End-to-end, one debate

1. **Intake.** Authenticated `POST /cases` with text or an uploaded file. The server mints the case id and session id — never the client. Returns a resumable case URL immediately.
2. **Guard.** Scope classifier, PII redaction pass, injection screen, per-user quota check. Rejections are explained in-band and never disconnect the socket.
3. **Structure.** One structured-output call extracts the case object. It is rendered for the user to confirm or correct. Nothing expensive has run yet.
4. **Research fan-out.** Section identification, precedent retrieval, evidence classes and procedural posture resolve concurrently via `asyncio.gather`. One wall-clock slot instead of four.
5. **Context assembly.** A single immutable case-context block is built once, registered with the provider's context cache, and reused verbatim by every subsequent turn.
6. **Rounds.** Each counsel drafts against the shared context plus a rolling debate summary, self-checks citations against the retrieved shortlist, and streams. Each turn is persisted the moment it completes.

7. **Bench.** The judge reads the full transcript and returns a structured ruling with confidence and the facts that would change it.
8. **Close.** Brief rendered, evidence gaps computed, case indexed for search, transcript retained under the user's stated retention policy.

Reliability and security posture

RELIABILITY

- Every step idempotent and separately retryable; a debate resumes from its last committed turn rather than restarting
- Circuit breaker per provider, with a documented degraded mode: retrieval-only results plus a plain statement that argument generation is unavailable
- Timeouts and bounded concurrency on every external call
- Dead-letter queue for failed debates, with an operator replay path

SECURITY

- Auth on every route that spends money or touches a case; socket handshake carries the JWT and the server binds the room to the user id
- Server-issued opaque session ids; membership checked on `join_room`
- Short access token + refresh token; revocation list in Redis so logout means something
- Per-user and per-IP rate limits, plus a hard monthly token budget per account
- Field-level encryption on case text; documented retention and a working delete
- Secrets from a manager, not `.env` baked into an image; CORS from config only — drop the hardcoded production origins in `socket_instance.py`

§ 06

Context management

Today there is none: no conversation memory, no summarisation, no prioritisation, and the full IPC section blob is re-sent on every turn.

Four tiers, assembled per turn

TIER	CONTENTS	LIFETIME	BUDGET
Immutable case context	Confirmed case object, section text, retrieved precedent shortlist, evidence classes	Whole debate	`~2.5k tok` `cached, sent once`
Rolling debate state	Structured ledger of claims: who raised what, what was conceded, what stands unanswered	Grows per turn	`~600 tok` `capped`
Recent verbatim	Last two turns in full — the current exchange must be exact, not summarised	Sliding window	`~800 tok`
Turn instruction	Persona, phase, output schema, the specific point being answered	This turn only	`~300 tok`

Prioritisation under pressure

When the budget is tight, drop in a fixed order and say so in the trace: full judgment text first (keep the headnote), then older precedent beyond the top two, then the mid-debate verbatim, then older ledger entries — *never* the section elements, the last opposing turn, or the output schema. A debate that forgets the section it is arguing is worse than a short one.

Compaction that preserves the argument

Generic summarisation destroys debates, because what matters is not what was said but what was *answered*. Compact into a structured claim ledger instead — each entry a claim, its author, its supporting citations, its status (`STANDING` `CONTESTED` `CONCEDED`), and the turn that last touched it. It compresses harder than prose, stays machine-checkable, and it is what the Bench needs at the end anyway. Trigger compaction on a token threshold, not a turn count.

Long-lived and cross-case memory

- **Within a case:** the claim ledger plus the confirmed structure is the complete resumption state. Reopening a case a week later reconstructs the context exactly, without replaying the transcript.
- **Across a user's cases:** stated preferences only — jurisdiction, language, the side they usually argue, output verbosity. Never facts from one matter carried into another; that is both a confidentiality breach and a correctness hazard.
- **Global:** only the verified corpus. This is the boundary F-01 and F-02 were crossing.

§ 07

Off-topic, abuse, injection, failure

A legal product attracts inputs a general assistant never sees — real allegations, real names, and users in distress. The current guard is one LLM call and a socket disconnect.

Scope, handled gracefully

Today a non-legal prompt emits an error and then `sio.disconnect()`s the user, who must reload the page. Replace the binary gate with a cheap classifier over four outcomes: *in scope* · *adjacent* (civil, family, consumer — say what is and isn't covered, offer the nearest thing) · *out of scope* (redirect with two example filings) · *needs a human now*. Never drop the connection; the session is the user's work.

That fourth branch matters more here than anywhere else in the product. Someone describing ongoing danger, or a self-harm risk in a case narrative, gets an interstitial with Indian helpline numbers before any debate runs. This is not an edge case in a criminal-law product — it is a Tuesday.

Prompt injection

Untrusted text now arrives from three directions: the user's description, uploaded documents, and retrieved corpus chunks. Defences, in order of value:

- **Structural separation.** User and document content goes in a typed field or a fenced, clearly-labelled block that the system prompt explicitly declares non-authoritative — never spliced into the instruction body as it is now.
- **Least privilege.** The debate path has no tools, no write access, no network. The worst an injection achieves is a bad argument, not an action.
- **Output validation.** Every turn is parsed against its schema and every citation matched against the retrieved shortlist before emission. An injected instruction to “cite *Sharma v. State*” fails the citation check regardless of how the model was persuaded.
- **Canary on the system prompt.** A known sentinel string that must never appear in output; if it does, drop the turn and alert.
- **Corpus provenance.** Only the ingest job writes, and it strips instruction-shaped text. F-01 was, precisely, a persistent injection vector.

Abuse and misuse

VECTOR	CONTROL
• Quota drain on open LLM routes • Room enumeration to read others' cases • Using counsel personas as a general jailbroken assistant • Seeking help to commit or conceal an offence • Bulk automated case generation	• Auth + per-user token budget + per-IP limit; delete `/gemini/*` • Server-issued session ids bound to the user; membership check on join • Persona prompts scoped to the case object; refuse out-of-case instructions • Intent classifier distinguishing *defending against* from *planning*; refuse, log, and offer the legitimate framing • Verified email, progressive limits, anomaly alerting on volume

Failure and edge cases

CASE	TODAY	TARGET
Model returns unparseable JSON	`None` propagates to a `TypeError` in a distant frame	Structured output; on failure, one reformat retry, then a typed error surfaced to the step
Rate limit / 429	3 blocking retries at 2s, then `None`	Async exponential backoff with jitter, provider fallback, honest queue-position message
Retrieval empty	Fabricates precedent and persists it	Argue from statute alone, labelled “no precedent found” — the honest answer is a feature
Two-word input	Full 12-call pipeline on nothing	Readiness check before spend; ask for the minimum facts first
Multi-section offence	Hard-coded to `[0]`, silently drops the rest	Rank all sections, debate the primary, list the others as also-charged
User disconnects mid-debate	State deleted, work lost, tokens burnt	Worker continues, turns persist, transcript waits at the case URL
Token expires mid-debate	UI still shows logged-in; save silently 401s	Silent refresh; save queued and retried after re-auth
Backend restart	Every in-flight debate lost	Redis-backed state; workers resume from the last committed turn

§ 08

Cost and latency, without giving up quality

The current pipeline is 12 sequential API round trips plus 4 seconds of deliberate `sleep`. Most of that is recoverable without touching output quality — and some cuts improve it.

LEVER	MECHANISM	EFFECT
Parallel research fan-out	Steps 04–06 are independent given the section. `asyncio.gather` instead of three awaits in a row.	−3 round trips of wall time
Drop the theatre	Two `sio.sleep(2)` calls exist to fake thinking. Real token streaming replaces them.	−4.0 s
Provider context caching	The case-context block is byte-identical across all three argument turns and is re-sent in full each time.	~65% fewer full-rate input tokens
Trim the section blob	The `full_data` IPC object is interpolated whole into every argument prompt. Counsel needs elements, mens rea, punishment range and defences — not procedural steps or the challenges list.	~40% smaller per argument prompt
Semantic result cache	Section descriptions and evidence classes are pure functions of a section number and change roughly never. Cache them in Redis permanently; today they are regenerated on every debate.	−2 calls after first use
Key lookups stop embedding	`search_by_ipc_section` embeds a query to fetch by exact metadata match. Use a Qdrant scroll filter.	−1 embed per lookup
Model routing	Two named tiers, both pointing at Flash-Lite by default. The seam exists so a deployment with paid quota can raise the Bench alone; on the free tier it stays a no-op.	Free tier carries a full hearing
Structured output	Schema-constrained generation stops the model narrating around its JSON, and removes the retry-the-whole-call path when parsing fails.	Fewer output tok fewer retries
Batch the ingest	Corpus embedding moves offline into batched jobs, off the request path entirely.	−4 embeds per debate

Net: roughly **12 request-path API calls down to 4**, first token visible in **2–4 seconds** instead of a 30–50 second wait for everything, and a *longer* debate — four rounds and a ruling — for materially less spend per case than one round costs today. Quality goes up because the budget moves from re-sending the same context twelve times to actually arguing.

Add a cost ceiling per debate in the orchestrator, log tokens per step against a debate id, and put spend-per-case on a dashboard. You cannot optimise what you are not measuring, and right now nothing counts tokens.

§ 09

What to adopt, and what to keep hand-rolled

Recommended where the library earns its dependency. Resisting a framework is also a decision.

> CONSTRAINT
>
> Everything below is free software that runs locally. **Gemini is the only external service**, and both model tiers are set to `gemini-flash-lite-latest` — the one tier whose free quota carries a whole hearing. No managed database, no commercial legal database, no hosted observability, no CDN or WAF subscription.

NEED	ADOPT	NOTE
Debate orchestration	**LangGraph**	The debate is literally a cyclic state graph with persistence and interrupts. Its checkpointer gives resumability and the *Object* control almost free. This is the one framework genuinely worth adding.
Background work	**asyncio tasks**, then **ARQ**	Once nothing blocks the loop, an in-process task per hearing is sufficient and durable because state is checkpointed. ARQ (MIT, Redis-only) is the step up when hearings need to outlive the web process.
Session & queue state	**Redis**	Replaces `case_data_store`, backs the Socket.IO manager for multi-replica, holds caches and quotas. One addition, four problems solved.
Retrieval quality	**Deterministic reranker**, then **bge-reranker-v2-m3**	Similarity blended with section overlap, court seniority and recency — transparent, unit-tested, no model weights. The cross-encoder is an Apache-2.0 local upgrade behind the same function when you want it.
Document parsing	**Gemini native PDF/image**, then **Docling**	The native path covers the common case with no extra service. Docling (MIT, runs locally) handles scans and tables where it struggles.
Rate limiting	**In-house, over the store**	Fixed-window counters on the same Redis-or-memory interface — no dependency, and multi-replica correct as soon as Redis is present. Closes F-05.
Observability	`/metrics` + **Prometheus** + **Grafana**	Prometheus exposition written by hand — no dependency, nothing leaves the machine. Tokens per step, circuit trips, and *citations stripped*, which is the product's health in one series.
Prompt regression	**stdlib** `unittest` + fixtures	Expected sections and retrieval as ordinary test cases, run in CI. No evaluation service, and the suite runs anywhere Python does.
Structured output	**Native** `response_schema`	Already in `google-genai`. Don't add Instructor or Outlines for this; the SDK does it.
Document rendering	**WeasyPrint** → headless Chrome → HTML	Server-side HTML → PDF, degrading through options that are all already present. DOCX is written as OOXML with the standard library rather than adding a dependency.

NEED	ADOPT	NOTE
TLS & edge	**Caddy**	Automatic Let's Encrypt certificates, security headers and per-path throttling — an Apache-2.0 container in place of a managed load balancer or CDN subscription.
Secrets	`.env` **/ Docker secrets**	No key-management service needed at this size. Rotating `JWT_SECRET_KEY` invalidates every session, which is the intended lever.
Auth	**Keep hand-rolled**	Fix status codes, add refresh and revocation. The current implementation is small and nearly right — a full identity provider is premature.
Vector store	**Keep Qdrant**	Correct choice. Add named vectors for hybrid search, payload indexes, and snapshots. No reason to move.
Corpus	**SC judgment portal** · **eCourts** · **India Code** · **Gazette**	The free official publishers, deliberately in place of any commercial legal database. Respect terms of use and rate limits; store provenance and a retrieval date per document. This is the actual asset.

§ 10

Impact against complexity

Fixes (F-nn) and features (01–10) on one scale. Everything in the upper-left ships first.

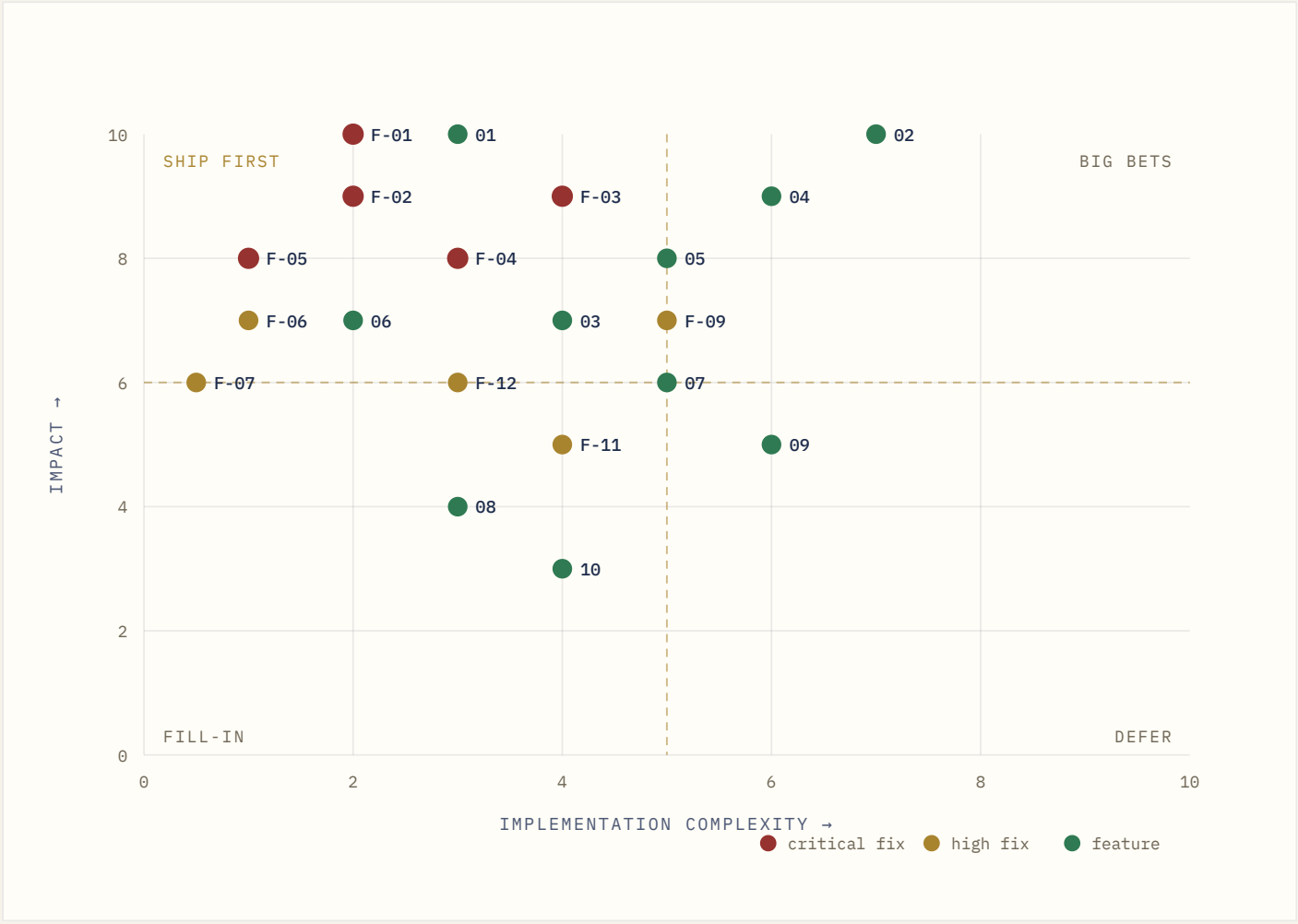

FIG. 2 — FIXES CLUSTER UPPER-LEFT BECAUSE THEY ARE SMALL AND LOAD-BEARING. FEATURES 02, 04 AND 09 ARE THE ONLY GENUINE INVESTMENTS.

F-01	hallucination loop	**F-02**	corpus leak	**F-03**	blocking event loop
F-04	room hijack	**F-05**	open LLM routes	**F-06**	evidence unused
F-07	analysis panel dead	**F-09**	retrieval quality	**F-11**	shared dict
F-12	injection surface	**01**	the Bench	**02**	verified citations
03	multi-round streaming	**04**	document intake	**05**	BNS dual-code
06	evidence gaps	**07**	take a side	**08**	export brief
09	strength meter	**10**	Indian languages		

The shape of this plot is the argument for the sequencing below: the five critical fixes are collectively about a week of work and unblock everything, while three of the ten features (the Bench, evidence gaps, export) are each a couple of days and transform how finished the product feels.

§ 11

Prototype → MVP → production

Four phases with exit criteria. Durations assume one or two engineers; the gates matter more than the weeks.

PHASE 0

Stop the bleeding

~1 week

- Delete every `save*` call from the generation path. Wipe the poisoned collections. (F-01, F-02)
- Auth on `/user/start-case`; server-issued session ids; JWT on the socket handshake; membership check in `join_room`. (F-04)
- Delete or gate the `/gemini/*` router; add rate limits everywhere. (F-05)
- Switch to `client.aio`, replace `time.sleep` with `asyncio.sleep`, parallelise steps 04–06. (F-03)
- Structured output with `response_schema`; delete the fence-stripping block. (F-14)
- Feed the user's evidence into the prompts. Emit `case_details`. Two small fixes, immediately visible. (F-06, F-07)
- Mongo indexes; correct auth status codes; uniform failure messages.

> **GATE**
> No path from model output to the vector store. Two concurrent debates complete without blocking each other. No unauthenticated endpoint spends money.

PHASE 1

Make it true

~3–4 weeks

- Build the ingest pipeline and load a real corpus — start with a single well-covered domain (say, offences against the person) rather than all of Indian criminal law.
- Rewrite retrieval: full-description queries, `k=8` → rerank → top 3, payload indexes, court as a boost not a filter. (F-09)
- Citation verification against the retrieved shortlist; unsupported claims labelled in the UI. (Feature 02)
- The Bench. Structured ruling with confidence and the facts that would flip it. (Feature 01)
- Redis: sessions, Socket.IO manager, caches. Kill `case_data_store`. (F-11)
- Prompt hardening — delimiter discipline, output canary, injection screen. (F-12)
- JSON logging with correlation ids; a `/metrics` endpoint; token accounting per debate id.
- The prompt regression suite, in CI, before any further prompt work.

> **GATE**
> Every citation in a transcript resolves to a real judgment in the corpus. Ten hand-checked cases return the correct section. Traces attribute latency and tokens per step.

PHASE 2

Make it worth returning to

~5–7 weeks

- LangGraph orchestration: durable, resumable, interruptible. Multi-round phased debate with token streaming; retire the artificial sleeps. (Feature 03)
- The four-tier context system and the claim ledger. Provider context caching on the immutable block.
- Document intake — FIR, chargesheet, notice → confirmed case object. (Feature 04)
- Evidence gap analyser and case brief export. (Features 06, 08)
- BNS/BNSS/BSA concordance, curated and stored, with date-based routing. (Feature 05)
- Frontend cleanup: one icon system, code splitting, dead components removed, the localStorage auth desync fixed. (F-13, F-15)

> **GATE**
> A debate survives a backend restart. First token under 4 seconds. A user can open a case from a week ago and continue it. Cost per debate is on a dashboard.

PHASE 3

Production

~6–8 weeks

- Take a side / moot-court mode and the calibrated strength meter, back-tested against labelled verdicts. (Features 07, 09)
- Indian-language intake and output. (Feature 10)
- Self-hosted Mongo, Qdrant and Redis with scripted, *rehearsed* restores; rolling redeploys; replicas behind Redis-backed sessions.
- TLS via Caddy, secrets in `.env`/Docker secrets, edge throttling, field-level encryption on case text, working delete and retention policy.
- Legal review: not-legal-advice notices where they will actually be read, terms, DPDP-aligned privacy policy, and a documented incident path.
- SLOs with alerting — debate success rate, p95 first token, citation verification rate, cost per case.

> **GATE**
>
> Load test at 50 concurrent debates within budget. Restore rehearsed from backup. Security review passed. Every user-visible number traceable to a source.

> **IF YOU ONLY DO ONE THING**
>
> Phase 0 is roughly a week and changes the product's category. Right now VaadVivaad manufactures legal citations and stores them as fact — everything else on this page is secondary to that. Cut the write path, load real judgments, and add the Bench, and you have something a law student would use on purpose rather than once.